Budget Buddy Student Expense Tracker

- **Task 1: Create the Test Plan**

Test Case	Scenario Type	Test Data (amount budget)	Description	Expected Result	
1	Happy Path	(50, 200)	Student makes a normal, affordable purchase within budget	True (Pass)	
2	Edge Case (Invalid Input)	(-10, 200) and (0, 200)	Student enters negative value or zero, which is invalid spending	False (Pass if correctly rejected)	
3	Edge Case (Over Budget)	(300, 200)	Student tries to spend more than available monthly budget	False (Pass if correctly rejected)	

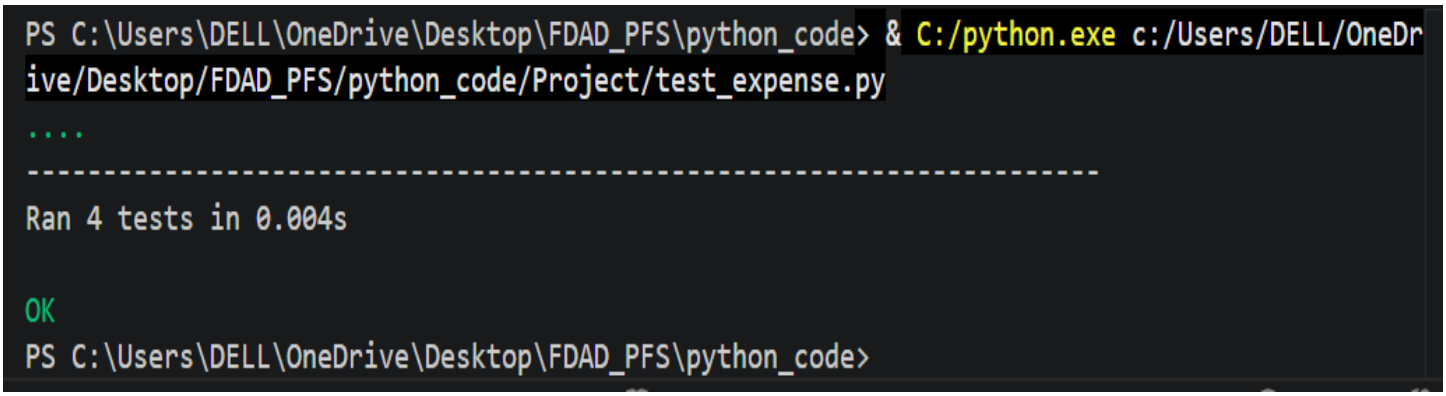
```python
def validate_expense(amount, budget):

# Rule 1: Check if the expense is a positive number

 if amount <= 0:
  return False

# Rule 2: Check if the user has enough budget remaining

 if amount > budget:
  return False

# If both rules are passed, allow the entry
 return True
```

- **Task 2: Writing Unit Tests**

```python
import unittest
from expense import validate_expense

# Test 1: Testing a normal transaction

class TestExpenseValidation(unittest.TestCase):

    # Test Case 1: Normal Expense (Happy Path)
    def test_normal_expense(self):
        self.assertTrue(validate_expense(50, 200))

    # Test Case 2: Negative Amount (Edge Case)
    def test_negative_expense(self):
        self.assertFalse(validate_expense(-10, 200))

    # Extra Edge Case: Zero Amount
    def test_zero_expense(self):
        self.assertFalse(validate_expense(0, 200))
```

```
    # Test Case 3: Over Budget (Edge Case)
    def test_over_budget(self):
        self.assertFalse(validate_expense(300, 200))


if __name__ == "__main__":
    unittest.main()
```

✓ Test Report: A screenshot of your terminal showing the "OK" status after running the tests.



- **Task 3: Write explanations for the outputs of each test case**

When running the unit tests, a dot (.) means the test has passed successfully, while an `F` indicates a failure where the actual result did not match the expected result.

The final summary shows the total number of tests executed and whether they all passed. If the output shows **"OK"**, it means all tests passed and the function is working correctly. If it shows **"FAILED"**, then at least one rule in the code is incorrect and needs to be fixed.

Each test case checks a financial rule in the system. The normal expense test ensures that valid spending within the budget is allowed. The negative or

zero expense test ensures that invalid inputs are rejected. The over-budget test ensures that a user cannot spend more than their available allowance.

Overall, the tests confirm that the expense validation system correctly enforces financial rules and prevents invalid or excessive spending.